Introduction to Generative AI

Generative AI (M25DES322) · Unit 1 · Exam Prep Notes (easy language)
Based on the Unit 1 slides (Dr. Rajeev Ranjan). Parts marked "extra" are short additions where a slide had only a table image or a heading.

> **Unit 1 at a glance:** What is GenAI & types → Evolution (rule-based → N-gram → RNN/LSTM → GAN/VAE → Transformer → Foundation models/LLMs) → Discriminative vs Generative → Probability & Information theory (entropy, cross-entropy, KL, MI, NLL) → Unsupervised & Self-supervised learning → Evaluation metrics (IS, FID, KID, MMD, CLIP, BLEU...) → Prompt engineering.

1. Introduction to Generative AI

- **Generative AI (GenAI)** is a branch of AI that **creates new content**: text, images, audio, video, software code, designs and synthetic data.
- It **learns patterns from large datasets** and uses them to generate new content that resembles the training data.
- **"Generative"** = the model can produce something **new**. It does not just look up a stored sentence. Example: after learning millions of sentences, it can write "AI technologies are increasingly being used to improve medical diagnosis" even if that exact sentence was never in the training data.

> **Example (plant leaf):** Traditional AI says "The leaf has Coffee Leaf Rust." Generative AI can also write a detailed explanation of the disease, give a treatment recommendation, produce a diagnostic report, generate Python analysis code, or create a synthetic image of an infected leaf.

Traditional AI vs Generative AI (important)

Aspect	Traditional AI	Generative AI
Purpose	Prediction / decision	Content generation
Output	Class, label, prediction	Text, image, audio, video, code
Typical question	"What is this?"	"Create something like this."
Learning	Patterns for a specific task	Broad patterns from large datasets
Example	Spam / not spam	Generate an email
Image use	Detect disease	Generate / synthesize an image
Language use	Sentiment classification	Generate an article
Interaction	Often task-specific	Natural-language interaction

Major types of Generative AI

Type	What it does
Text generation	Articles, reports, emails, summaries, stories, Q&A answers, research drafts, documentation. Main technology: **Large Language Models (LLMs)**.
Image generation	Creates images from text descriptions (prompt → image generation → generated image).
Audio & speech	Human-like speech, music, sound effects, voice conversion, speech translation, audiobooks, virtual assistants, accessibility, education, voice-based healthcare.
Video generation	From text prompts, images, existing videos or scripts; used in education, advertising, film, simulation, training.
Code generation	Python, Java, C/C++, JavaScript, SQL, HTML/CSS, infrastructure code.
Multimodal generation	Systems that work across several modalities (text + image + audio + video + code).

2. Evolution of Generative AI (Evolutionary Trajectory)

Stage 1: Rule-based and statistical models (pre-2010s)

- **Hand-crafted rule-based systems:** e.g. IF user says "Hello" THEN reply "Hello! How can I help you?". Grammar rules like Sentence → Noun Phrase + Verb Phrase.
  - Characteristics: human-defined rules, deterministic, little or no learning, hard to scale, limited handling of language variation.
  - Limitation: cannot handle situations the programmer did not anticipate.
- **N-gram models:** statistical language modelling. An N-gram is a sequence of N words. Unigram: one word (Artificial); Bigram: two (Artificial Intelligence); Trigram: three (Artificial Intelligence Education). Predicts the next word from a few previous words: bigram $P(w_t \mid w_{t-1})$, trigram $P(w_t \mid w_{t-2}, w_{t-1})$. **Limitation:** limited context window and data sparsity.
- **Markov chains:** probabilistic transitions between states, $P(X_{t+1} \mid X_t)$; e.g. Sunny → Cloudy → Rainy. Future depends mainly on the current state. Useful but weak for complex, long-range dependencies.

Stage 2: Deep learning and recurrent architectures (2010 to 2014)

- **Neural networks** learn numeric representations of words (instead of storing probabilities like "student" → "reads" = 0.35). Led to neural language models and RNNs.
- **RNN:** designed for sequential data. Hidden state: $\mathbf{h_t = f(W_x x_t + W_h h_{t-1} + b)}$ (W_x = input-to-hidden weights; W_h = recurrent weights from previous hidden state; b = bias; h_{t-1} = previous hidden state). Uses: language modelling, speech recognition, machine translation, time-series forecasting, text generation. **Problem:** vanishing gradient, poor long-term dependencies.
- **LSTM and GRU:** gated architectures that control what is **remembered, forgotten, updated and output**; much better for long dependencies.
- **Attention mechanism:** a big breakthrough. Instead of one compressed hidden state, the model decides **which parts of the input matter** now. Example: "The student submitted the project because *it* was completed": attention links "it" to "project".

Stage 3: The generative explosion (2014 to 2017)

- **GANs (2014):** competitive framework of **Generator vs Discriminator**; revolutionized realistic image generation.
- **VAEs (2013-2014):** probabilistic approach to mapping data into a continuous **latent space**.
- Modern GenAI generates: Text + Image + Audio + Video + Code.

Stage 4: Transformer era and scaled pre-training (2017 to present)

- **Transformer (2017):** uses **self-attention** to model relationships among tokens, instead of processing one step at a time like an RNN. This removed the sequential bottleneck and allowed **parallel computation** on huge datasets.
- Benefits: better contextual understanding, long-range relationships, highly parallel training, scaling to very large models, strong NLP performance. It is the foundation of modern LLMs.
- **Foundation model:** trained on broad datasets, then **adapted to many tasks** (instead of one model per task). Shifted AI from task-specific to **general-purpose pretrained models**.
- **LLM:** large neural model trained on huge datasets to learn patterns in language and related data.

Evolution summary table (very important)

Gen	Technology	Context / memory	Main capability
1	Rule-based	Explicit rules	Decision / response
2	N-gram	Few preceding tokens	Local language prediction
3	Markov	Previous state / history	Probabilistic sequence modelling
4	RNN	Hidden state	Sequential learning
5	LSTM / GRU	Gated memory	Long-term dependencies
6	GAN / VAE	Latent representation	Data generation
7	Transformer	Attention	Rich contextual modelling
8	Foundation models	Large-scale learned representation	General-purpose AI
9	LLM / GenAI	Large context + generation	Content creation
10	Multimodal / Agentic AI	Multiple modalities + tools	Generation, interaction, task execution

3. Discriminative vs Generative Models

Discriminative models

- Learn the relationship between input X and output Y. They model the **conditional probability P(Y | X)**.
- Prediction: $\hat{Y} = \text{argmax}_Y P(Y \mid X)$: "Given an input, what is the most likely class?" Example: X = email text, $Y \in$ {Spam, Not Spam}.
- **Examples:** Logistic Regression, SVM, Decision Trees, Random Forest, classification neural networks, CNN classifiers, BERT used for classification.

Generative models

- Learn the **underlying probability distribution of the data** and use it to **generate new data**. They model **P(X)** or the **joint distribution P(X, Y)**.
- Example: trained on "The cat is sitting on the mat", "The dog is playing in the garden", "The boy is reading a book", a generative language model can produce "The girl is reading in the library."
- **Examples:** Naive Bayes, HMMs, GMMs, VAEs, GANs, Diffusion models, GPT.

Feature	Discriminative	Generative
Main purpose	Classification / prediction	Generate new data
Learns	P(Y | X)	P(X), or joint P(X, Y)
Main question	"Which class?"	"What could the data look like?"
Generates new samples?	Generally no	Yes
Classification	Excellent use case	Can also classify
Text / image generation	Not its primary purpose	Major application
Examples	SVM, Logistic Regression, classifiers	GAN, VAE, Diffusion, GPT

4. Basic Probability

- **Role of probability in GenAI:** modelling data distributions; estimating likelihood of data points (e.g. VAEs train to maximize the likelihood of observed data); priors and likelihoods in Bayesian models; loss functions like **negative log-likelihood and KL divergence**; quantifying uncertainty (VAEs encode data into a probabilistic latent space; the GAN discriminator estimates the probability a sample is real or fake).
- **Common distributions:** uniform, binomial, Bernoulli, Poisson, normal, t, exponential.
- **Conditional probability (discriminative):** $P(A|B) = P(A \cap B) / P(B)$. **Bayes theorem:** $P(A|B) = P(B|A)\, P(A) / P(B)$.
- **Joint probability (generative):** $P(x, y) = P(A \text{ and } B) = P(A \cap B)$.
- **Random variable:** a discrete variable X takes values x with probability mass function $P(X = x)$. **Axioms:** $0 \leq P(x) \leq 1$ and $\Sigma P(x) = 1$.
- **Joint and conditional:** $P(X, Y) = P(Y \mid X)\, P(X)$.
- **Chain rule:** LLMs factorize the probability of a token sequence one token at a time: $\mathbf{P(w_1, \ldots, w_T) = \prod P(w_t \mid w_1, \ldots, w_{t-1})}$.
- **Expectation:** $E_{X \sim P}[f(X)] = \Sigma\, P(x)\, f(x)$ (or an integral for continuous X).

5. Information Theory

Information theory studies how information is represented, measured and transmitted. Key concepts in GenAI: **Entropy, Cross-entropy, KL divergence, Mutual information, Negative log-likelihood.**

5.1 Entropy

- Entropy measures the **uncertainty** in a probability distribution.

Shannon entropy: $H(X) = -\Sigma P(x) \log_2 P(x)$

Example	Calculation	Meaning
A = 0.5, B = 0.5	$H = -(0.5 \log_2 0.5 + 0.5 \log_2 0.5) =$ **1 bit**	Maximum uncertainty between two equally likely choices
A = 0.9, B = 0.1	$H = -(0.9 \log_2 0.9 + 0.1 \log_2 0.1) \approx$ **0.469 bits**	Lower entropy: model is more certain

- **High-probability** events ("The sun rose today") carry **very low information**; **low-probability** events ("It rained in the Sahara") carry **high information**.

Entropy and temperature

- **Temperature** controls the randomness of token generation: $\mathbf{P_i = \exp(z_i/T) / \Sigma_j \exp(z_j/T)}$ (z_i = model logit, T = temperature).
- "The capital of France is ...": **low T** → almost always "Paris" (sharp distribution). **High T** → flatter distribution, more chance of less-probable tokens (more random/creative).

5.2 Cross-entropy

$H(P, Q) = -\Sigma P(x) \log Q(x) = E_{X\sim P}[-\log Q(x)]$
Average number of bits needed to identify an event from true distribution **P** when using a code built for the model's distribution **Q**.

- **Role in GenAI:** primary **loss function** for training autoregressive LLMs (next-token prediction) and classifiers.
- Example: correct token is A. If model predicts P(A) = 0.8, loss = −log(0.8) ≈ **0.223**. If it predicts P(A) = 0.1, loss = −log(0.1) ≈ **2.303**. **Low probability to the correct answer gives a large loss.**
- Training sentence "The cat sleeps." gives pairs: The → cat; The cat → sleeps; The cat sleeps → "."
- **Training objective:** $L = -(1/N) \Sigma_{t=1..N} \log P_\theta(x_t \mid x_{<t})$ (next-token cross-entropy).

5.3 KL divergence

$D_{KL}(P \,||\, Q) = \Sigma P(x) \log(P(x) / Q(x))$

- Measures **how one distribution differs from another**; the information lost when Q is used to approximate P.
- **Not symmetric:** $D_{KL}(P||Q) \neq D_{KL}(Q||P)$. So it is a divergence measure, not a true distance metric.
- **Example:** P = [0.8, 0.2], Q = [0.6, 0.4]: $D_{KL} = 0.8 \ln(0.8/0.6) + 0.2 \ln(0.2/0.4) \approx$ **0.0915** (natural log). A value of **0** means the two distributions are identical.
- **Properties: non-negative** ($D_{KL} \geq 0$, equal to 0 only if P = Q; Gibbs' inequality); **asymmetric**.
- **Relationship: $H(P, Q) = H(P) + D_{KL}(P \,||\, Q)$**. Since H(P) (data entropy) is fixed, **minimizing cross-entropy minimizes KL divergence**.

One-line summary: Probability says what is likely; entropy measures uncertainty; cross-entropy measures prediction error; KL divergence measures how different two distributions are.

5.4 Mutual information (MI)

- Amount of information one random variable contains about another.
- **$I(X; Y) = \Sigma_{x,y}\ p(x,y)\ \log(\ p(x,y)\ /\ (p(x)\ p(y))\ )$**. For continuous variables, the sum becomes an integral. p(x,y) = joint, p(x), p(y) = marginals.
- If X and Y are **independent**, MI = **0**.
- In GenAI: relation between input data, latent representations, generated outputs and learned features. Example: prompt "A red car on a beach" → AI model learns semantic features → generated image. **Higher MI:** image reflects the prompt well. **Lower MI:** image misses prompt details. (High MI alone does not guarantee a visually good image.)

5.5 Negative log-likelihood (NLL)

- Measures **how poorly a model assigns probability to the actual observed data**. **Lower NLL = better** predictions.
- **$NLL(x) = -\log p_\theta(x)$**. For N samples: **$NLL = -(1/N)\ \Sigma\ \log p_\theta(x_i)$** ($x_i$ = observed sample; p_θ = probability from model; θ = parameters).

"The sky is ___" (correct = blue)	Probability	NLL = −ln(p)
blue	0.90	0.105
cloudy	0.50	0.693
green	0.01	4.605

Higher probability for the correct token means lower NLL; a very low probability gives a much larger penalty.

6. Unsupervised and Self-Supervised Learning

- Both learn representations from **unlabeled data**, avoiding costly human labelling. Self-supervised learning (SSL) is effectively modern unsupervised learning at massive scale; it is the **foundation pre-training step** for LLMs, diffusion models and vision transformers.

Unsupervised learning

- Model learns patterns/structure from **unlabeled data**; there is **no predefined target label y**.
- **Techniques:** Clustering (K-Means, DBSCAN, hierarchical); Dimensionality reduction (PCA, t-SNE, UMAP); Density estimation; Association rule learning; Generative modelling.
- **In GenAI:** generative models learn the data distribution P(X) (e.g. from thousands of face images), then generate a new sample $X_{new} \sim P(X)$. The new face need not be an exact copy of a training image. GAN example: z → G(z) → generated image.

Self-supervised learning

- The model **creates its own training targets from the input data**: input data → automatically generated target. No human labelling.

- **Hiding a word:** "The cat is sitting on the ___" (answer "mat" comes from the original sentence).
- **Next-token prediction (GPT):** "Artificial intelligence is" → "transforming". Objective: $L = -\Sigma_t \log P_\theta(x_t \mid x_{<t})$.
- **Masked prediction (BERT):** "The capital of India is [MASK]" → "New Delhi".
- **Difference:** GPT = previous tokens → next token. BERT = context with masked token → missing token.
- **GenAI pipeline:** Large unlabeled dataset → self-supervised pretraining → foundation model → fine-tuning → GenAI application. (e.g. internet-scale text → next-token prediction → LLM)

Unsupervised vs Self-supervised

Feature	Unsupervised	Self-supervised
Human labels	Not required	Not required
Target	Usually no explicit target	Target generated from the data
Main objective	Discover structure	Learn useful representations / predict hidden information
Example	K-Means	Next-token prediction
Data	Unlabeled	Unlabeled
GenAI relevance	High	Very high
LLM pretraining	Less typical description	Core approach
Example models	Clustering	GPT, BERT, MAE
Output	Cluster labels, low-dimensional projections, density estimates	General-purpose pretrained embeddings, fine-tuned or prompted for tasks

7. Evaluation Metrics for Generative Models

- These measure the **quality, diversity and fidelity** of generated data. Evaluation is hard because there is **no single ground-truth target**: a good model must look realistic and cover the full real distribution **without copying**.
- **Two key dimensions: Fidelity (quality)** = how realistic and believable each sample is. **Diversity (coverage)** = how well the full distribution is covered, without **mode collapse** (dropping modes) or repeating samples.

Computer vision and multimodal metrics

- **Why not pixel-level (MSE)?** Perceptually similar images can have very different pixel values. Example: the same cat shifted slightly right looks identical to a human, but MSE is high.
- Modern metrics first pass images through a **pretrained network (Inception-v3 or CLIP)** to get **embeddings** (feature vectors), then compare them using cosine distance or Fréchet distance.

Metric	Formula	Meaning / interpretation
Inception Score (IS)	$IS = \exp(E_x[D_{KL}(p(y\|x) \| p(y))])$ $p(y\|x)$ = class probabilities for image x; $p(y)$ = marginal class distribution over generated images	**High IS:** images individually recognizable (confident) and collectively diverse. **Low IS:** unrealistic or lacking diversity (blurry images, or all-dog repetitive images). **Limitation:** does not compare with real images.
Fréchet Inception Distance (FID)	$FID = \|\|\mu_r - \mu_g\|\|^2 + Tr(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2})$ (μ, Σ = mean and covariance of real (r) and generated (g) features; both assumed Gaussian)	**Lower FID = better.** Compares real vs generated feature distributions; measures image quality and diversity. More informative than IS because it explicitly compares real and generated data.
Kernel Inception Distance (KID)	$KID = MMD^2(X_r, X_g)$ in Inception feature space, with polynomial kernel $k(x,y) = (x^T y / d + 1)^3$ (d = feature dimension, often 2,048)	**Lower KID = better.** Unbiased estimator; can work better than FID with small sample sizes. The cubic kernel captures higher-order differences, not just mean and variance.
MMD (Maximum Mean Discrepancy)	$MMD^2(P,Q) = E[k(x,x')] + E[k(y,y')] - 2E[k(x,y)]$	**Small MMD:** feature distributions similar. **Large MMD:** different. Three parts: similarity among real, similarity among generated, similarity between real and generated (times −2). Compares feature distributions, not raw pixels.
CLIP-based	Similarity = $(E_{text} \cdot E_{image}) / (\|\|E_{text}\|\| \, \|\|E_{image}\|\|)$ (cosine similarity)	Measures **text-prompt vs generated-image alignment**. Higher similarity = better alignment. Used for DALL-E-type models, Stable Diffusion, text-to-image and captioning evaluation.

Text and language model metrics

Metric	Category	Mechanism	Application
Perplexity (PPL)	Probabilistic	How "surprised" the model is by text; exponential of average NLL (extra: lower is better)	Language model pre-training and fluency
BLEU	Surface n-gram	Modified n-gram precision, penalized for brevity against reference text	Machine translation
ROUGE (ROUGE-L)	Surface n-gram	Longest Common Subsequence (LCS) recall relative to reference summaries	Text summarization
BERTScore	Semantic similarity	Cosine similarity of contextual embeddings (BERT/RoBERTa) between tokens	General text generation (handles paraphrasing)
LLM-as-a-Judge	Semantic / preference	Strong frontier LLMs (e.g. GPT-4) score outputs using rubrics (MT-Bench, AlpacaEval, Arena-Hard)	Alignment, instruction following, conversational AI

8. Prompt Engineering Fundamentals

- **Prompt engineering** = designing, refining and structuring the inputs (prompts) given to an LLM/GenAI model to get **accurate, relevant, consistent and useful** outputs.
- **Why important:** helps the AI understand intent; improves quality and relevance; reduces ambiguity and unwanted information; controls format, style and detail; improves text generation, summarization, coding, reasoning and Q&A.

4 core pillars

Pillar	Meaning
1. Clear context and role assignment	Say who the model is acting as and why (e.g. "Act as a senior data analyst...")
2. Explicit instructions and constraints	Be specific on what to do and not do; set length, format, style, tone (e.g. "3-bullet summary without jargon")
3. Delimiters and structure	Use XML tags (<context>, <input>), triple quotes, or markdown headers to separate instructions from data
4. Task decomposition	Break complex multi-step work into smaller sequential steps

Characteristics of a good prompt

Clear, Specific, Contextual, Concise, Structured, Unambiguous, Output-oriented (state the desired format).

Types of prompts (with slide examples)

Technique	What it is	Example
Zero-shot	Task asked directly, **no examples**	"Translate 'artificial intelligence' into French."
One-shot	**One example** given to guide the model	"Python is a programming language." → "What is Python?" Now convert: "TensorFlow is a machine learning framework."
Few-shot	**Several input-output examples** before the real task	Great service! → Positive; Horrible delay. → Negative; The package arrived today. → Neutral
Chain-of-Thought (CoT)	Model shows **step-by-step reasoning** before the answer; good for math, logic, multi-step problems	Trigger: "Think step-by-step before answering."
Self-consistency CoT	Generate **multiple reasoning paths** and take the **majority vote**	20% discount on Rs 1,000, then 18% GST: final price?
Tree-of-Thoughts (ToT)	Explores **multiple branches** of reasoning at each step; for search and planning	Choose the best AI project approach in 4 weeks, considering time, complexity, outcome
Comparative	Compare two or more concepts	Compare CNN and RNN in a table (architecture, input, applications, advantages, limitations)
Role / system prompting	Set global instructions and behaviour before the user interaction; sets perspective, terminology, level	"You are an experienced cybersecurity instructor. Explain phishing to first-year students."
Generated knowledge	Ask the model to **first generate relevant facts**, then answer the main question	Why do deep learning models do poorly on plant disease detection in field conditions?
Directional stimulus	Give **hints/keywords** to steer the answer	Explain why stationarity is needed before ARIMA. Discuss: mean, variance, autocorrelation, differencing, forecasting
Structured prompt	Specifies an **output structure**	Explain cloud computing with: Definition, Characteristics, Service models, Deployment models, Applications
Extraction	Extract specific information from given content	Extract author names, year, dataset and accuracy from a research paper
ReAct (Reason + Act)	Combine reasoning with **external tool actions** (web search, database, API)	Model reasons, then calls a search tool, then continues
Negative prompting / safeguards	State disallowed topics/styles or hallucinated assumptions	"If you do not know the answer from the context, say 'I don't know'."
Contextual	Adds **background information** with the task	Students know Python but not neural networks; explain CNNs with a simple image-classification example

Constraint-based	Imposes restrictions on the output	Explain GenAI in exactly 150 words, simple language, three examples
Summarization	Condense information	Summarize the paper in 10 bullets: methodology, results, limitations

9. Last-Minute Revision

- **GenAI** creates new content (text, image, audio, video, code); traditional AI predicts/classifies.
- **Evolution:** Rule-based → N-gram → Markov → RNN → LSTM/GRU (+ attention) → GAN/VAE (2013-14) → Transformer (2017) → Foundation models → LLM → Multimodal/Agentic.
- **Discriminative:** P(Y|X), classifies. **Generative:** P(X) or P(X,Y), generates.
- **Entropy** $H = -\Sigma P \log P$ (uncertainty). **Cross-entropy** $H(P,Q) = -\Sigma P \log Q$ (LLM loss). **KL** = $\Sigma P \log(P/Q)$, non-negative, asymmetric. $H(P,Q) = H(P) + KL$. **MI** = shared information, zero if independent. **NLL** = $-\log p$.
- **Temperature:** low = deterministic; high = more random.
- **Unsupervised:** no labels, find structure. **Self-supervised:** labels made from the data (GPT next-token, BERT masked).
- **Metrics:** IS (high better), FID (low better), KID (low better), MMD (small better), CLIP (cosine similarity), BLEU (translation), ROUGE (summarization), BERTScore (semantic), Perplexity (language modelling), LLM-as-a-Judge.
- **Prompting:** zero-shot, one-shot, few-shot, CoT, self-consistency, ToT, role, ReAct, structured, constraint-based, etc. 4 pillars: role/context, instructions, delimiters, task decomposition.